Tabla de contenidos

1. Serialización de objetos
    a. JSON
        i. Serialización de Objetos Personalizados
            1. Personalizar la serialización en `json`
            2. Personalizar la deserialización en `json`
        ii. Errores en serialización

Serialización de objetos

Toda la información que almacena un computador se guarda en base a *bits* (ceros y unos) y *bytes* (secuencias de 8 *bits*). Esta semana tendremos un primer acercamiento al uso y manejo de *bytes* en Python.

Imaginemos que buscamos guardar una estructura de datos o una instancia de una clase para volver a leerla más adelante o para comunicarla a otro programa. De alguna forma, estos datos deben ser guardados como una serie o secuencia de *bytes*. Aquí es cuando aparece el concepto de **serialización**.

Este concepto se refiere al procedimiento de transformar un objeto en una secuencia o serie de *bytes*. Esto nos permite almacenar la información y el estado de un objeto de forma persistente, por ejemplo en un archivo o una base de datos para poder consultarlo más tarde. También nos permite enviar el objeto a otros computadores y programas.

JSON

JSON (JavaScript Object Notation) es un formato de texto estándar de intercambio de datos que puede ser interpretado por muchos lenguajes, y por ende es algo ampliamente utilizado para traspasar información de un programa a otro (por ejemplo, la comunicación entre dos computadores mediante internet). Además, JSON es *human-readable*, es decir, puede ser fácilmente leído y entendido por humanos.

El formato en que almacena la información es similar, pero no igual, a los diccionarios de Python.

En JSON solo es posible serializar instancias de `int`, `str`, `float`, `dict`, `bool`, `list`, `tuple` y `None`, de acuerdo a esta tabla de transformación que puedes revisar en este link. Por defecto no es posible serializar funciones o instancias de otras clases.

En Python, existe un módulo llamado `json` que provee métodos para serializar objetos en el formato JSON. Este módulo ofrece los siguientes métodos:

- `dumps`: serializa un objeto, es decir, lo **guarda** como un `str` en formato JSON.
- `loads`: deserializa un `str` en formato JSON, es decir, lo **carga** un objeto de Python.

Una vez que un objeto es serializado, este es persistente y está listo para volver a ser usado en el futuro por el mismo u otro programa.

Primero, veamos cómo se comporta JSON al serializar y deserializar una lista con distintos valores:

```
import json


lista = ["a", 1, 3, "hola"]
lista_serializada = json.dumps(lista)

print(f"Resultado serialización: {lista_serializada}")
print(f"Tipo de versión serializada: {type(lista_serializada)}")

lista_deserializada = json.loads(lista_serializada)
print(f"Resultado deserialización: {lista_deserializada}")

print()
print(f"Lista original: {lista}")
print(f"Lista deserializada: {lista_deserializada}")
print(f"¿Las listas son iguales? {lista == lista_deserializada}")
print(f"¿Las listas son el mismo objeto? {lista is
lista_deserializada}")

print("=" * 40)
print(type(lista))
print(type(lista_serializada))
```

```
Resultado serialización: ["a", 1, 3, "hola"]
Tipo de versión serializada: <class 'str'>
Resultado deserialización: ['a', 1, 3, 'hola']

Lista original: ['a', 1, 3, 'hola']
Lista deserializada: ['a', 1, 3, 'hola']
¿Las listas son iguales? True
¿Las listas son el mismo objeto? False
========================================
<class 'list'>
<class 'str'>
```

Pero hay que tener ojo con la transformación de algunos tipos de datos, ya que -como se explica en la tabla de transformación- algunos objetos de Python comparten un mismo tipo de dato en el formato JSON:

```
import json


tupla = (None, 1, 22.33, {"a": True, "b": False})
tupla_serializada = json.dumps(tupla)
```

```
print(f"Resultado serialización: {tupla_serializada}")
print(f"Tipo de versión serializada: {type(tupla_serializada)}")

tupla_deserializada = json.loads(tupla_serializada)
print(f"Resultado deserialización: {tupla_deserializada}")
print(f"Tipo de versión deserializada: {type(tupla_deserializada)}")

print()
print(f"Tupla original: {tupla}")
print(f"Tupla deserializada: {tupla_deserializada}")
print(f"¿Las tuplas son iguales? {tupla == tupla_deserializada}")
print(f"¿Las tuplas son el mismo objeto? {tupla is
tupla_deserializada}")
```

```
Resultado serialización: [null, 1, 22.33, {"a": true, "b": false}]
Tipo de versión serializada: <class 'str'>
Resultado deserialización: [None, 1, 22.33, {'a': True, 'b': False}]
Tipo de versión deserializada: <class 'list'>

Tupla original: (None, 1, 22.33, {'a': True, 'b': False})
Tupla deserializada: [None, 1, 22.33, {'a': True, 'b': False}]
¿Las tuplas son iguales? False
¿Las tuplas son el mismo objeto? False
```

Además, `json` nos ofrece los métodos `dump` y `load` (casi el mismo nombre que los anteriores, pero sin la **`s`** al final). Estos métodos también serializan y deserializan, pero **a través de archivos**:

- `dump`: guarda un archivo con el objeto serializado.
- `load`: deserializa un objeto almacenado en un archivo.

```
import json
from os import path


lista = [67, 68, 69, 65]

with open(path.join("data", "mi_lista.json"), "w") as archivo:
    json.dump(lista, archivo)

with open(path.join("data", "mi_lista.json"), "r") as archivo:
    lista_cargada = json.load(archivo)

print(f"Lista original: {lista}")
print(f"Lista cargada : {lista_cargada}")
print(f"¿Las listas son iguales? {lista == lista_cargada}")
print(f"¿Las listas son el mismo objeto? {lista is lista_cargada}")
```

```
Lista original: [67, 68, 69, 65]
Lista cargada : [67, 68, 69, 65]
```

```
¿Las listas son iguales? True
¿Las listas son el mismo objeto? False
```

Serialización de Objetos Personalizados

Dado que ahora entendemos un mejor, el la serialización y deserialización, puede que les surja la siguiente duda:

> ¿Y si guardo mi información en clases personalizadas? ¿Puedo serializar la información de otros tipos de objetos?

Como JSON solo permite serializar algunos tipos de objetos (`int`, `str`, `float`, `dict`, `bool`, `list`, `tuple` y `None`), si se quiere serializar la información de otros objetos, entonces habrá que adaptar la información para que sea compatible a lo que acepta JSON.

```python
from itertools import count
import json


class Mascota:
    id = count()

    def __init__(self, nombre: str, edad: int, especie: str) -> None:
        self.nombre = nombre
        self.edad = edad
        self.especie = especie
        self.id_ = next(self.id)

m = Mascota("Luna", 8, "Gato")

json_string = json.dumps(m.__dict__)
print("Datos en formato JSON:", type(json_string), json_string)

json_deserializado = json.loads(json_string)
print("Datos en formato Python:", type(json_deserializado),
json_deserializado)
```

```
Datos en formato JSON: <class 'str'> {"nombre": "Luna", "edad": 8,
"especie": "Gato", "id_": 0}
Datos en formato Python: <class 'dict'> {'nombre': 'Luna', 'edad': 8,
'especie': 'Gato', 'id_': 0}
```

Como podemos ver, esto ayuda al momento de serializar la información de una clase, pero no nos permite cargar su información como una instancia de la misma.

Para poder serializar y deserializar correctamente la información de objetos no compatibles con JSON será necesario personalizar la serialización y deserialización, para esto contamos con:

- `json.JSONEncoder`
- `object_hook`

Personalizar la serialización en `json`

Cuando `json` trata de serializar un objeto, utiliza la clase `JSONEncoder` para transformar la información a un formato compatible con JSON. Esta clase es **extensible**, es decir, podemos crear clases que hereden de ella y así permitir que otros objetos sean serializables con `json`.

Para lograr que nuestra clase que herede de `JSONEncoder` sea capaz de serializar la información, deberemos sobrescribir el método `default` y, como hicimos en el ejemplo anterior, transformar la información del objeto y llamar el método `default` por defecto con la información transformada.

```
import json
from datetime import datetime


class MascotaEncoder(json.JSONEncoder):

    def default(self, obj):
        """Serializa en forma personalizada el objeto de tipo
Mascota"""
        if isinstance(obj, Mascota):
            return {
                "Mascota_id": obj.id_,
                "nombre": obj.nombre,
                "edad": obj.edad,
                "especie": obj.especie,
                "ano_nacimiento": datetime.now().year - obj.edad,
            }
        # Mantenemos la serialización por defecto para otros tipos
        return super().default(obj)


# Creamos tres instancias
m1 = Mascota("Pepa", 70, "Tortuga")
m2 = Mascota("Cachirulo", 5, "Gato")
m3 = Mascota("Guillermo", 0, "Gato")
```

Probamos la serialización usando el método por defecto:

```
json_string = json.dumps(m1.__dict__)

print(json_string)
```

```
{"nombre": "Pepa", "edad": 70, "especie": "Tortuga", "id_": 1}
```

Ahora, comparemos el resultado al serializar usando nuestro método personalizado:

```
json_string = json.dumps(m1, cls=MascotaEncoder)
print(json_string)
```

```
json_string = json.dumps(m2, cls=MascotaEncoder)
print(json_string)

json_string = json.dumps(m3, cls=MascotaEncoder)
print(json_string)
```

```
{"Mascota_id": 1, "nombre": "Pepa", "edad": 70, "especie": "Tortuga",
"ano_nacimiento": 1956}
{"Mascota_id": 2, "nombre": "Cachirulo", "edad": 5, "especie": "Gato",
"ano_nacimiento": 2021}
{"Mascota_id": 3, "nombre": "Guillermo", "edad": 0, "especie": "Gato",
"ano_nacimiento": 2026}
```

También podemos extender el mismo `JSONEncoder` para que maneje **más de una clase personalizada**. Basta con agregar más ramas `isinstance` dentro del método `default`. Si el objeto no corresponde a ningún tipo conocido, la llamada a `super().default(obj)` levantará un `TypeError`, avisándonos que ese tipo no es serializable.

```
class Veterinario:
    def __init__(self, nombre: str, especialidad: str) -> None:
        self.nombre = nombre
        self.especialidad = especialidad


class MascotaVeterinarioEncoder(json.JSONEncoder):

    def default(self, obj):
        if isinstance(obj, Mascota):
            return {
                "tipo": "Mascota",
                "nombre": obj.nombre,
                "edad": obj.edad,
                "especie": obj.especie,
            }
        if isinstance(obj, Veterinario):
            return {
                "tipo": "Veterinario",
                "nombre": obj.nombre,
                "especialidad": obj.especialidad,
            }
        # Para cualquier otro tipo no soportado, levanta TypeError
        return super().default(obj)


v = Veterinario("Dra. Torres", "Exóticos")

# Serializar un Veterinario
print("Veterinario:")
print(json.dumps(v, cls=MascotaVeterinarioEncoder))
```

```python
# Serializar una Mascota con el mismo encoder
print("\nMascota:")
print(json.dumps(m1, cls=MascotaVeterinarioEncoder))

# Serializar una lista mixta
print("\nLista mixta:")
print(json.dumps([m2, v, m3], cls=MascotaVeterinarioEncoder,
indent=2))

# Intentar serializar un tipo no soportado → TypeError
try:
    json.dumps(object(), cls=MascotaVeterinarioEncoder)
except TypeError as e:
    print(f"\nTipo no soportado → {e}")
```

```
Veterinario:
{"tipo": "Veterinario", "nombre": "Dra. Torres", "especialidad": "Ex\
u00f3ticos"}

Mascota:
{"tipo": "Mascota", "nombre": "Pepa", "edad": 70, "especie":
"Tortuga"}

Lista mixta:
[
  {
    "tipo": "Mascota",
    "nombre": "Cachirulo",
    "edad": 5,
    "especie": "Gato"
  },
  {
    "tipo": "Veterinario",
    "nombre": "Dra. Torres",
    "especialidad": "Ex\u00f3ticos"
  },
  {
    "tipo": "Mascota",
    "nombre": "Guillermo",
    "edad": 0,
    "especie": "Gato"
  }
]

Tipo no soportado → Object of type object is not JSON serializable
```

Personalizar la deserialización en `json`

De forma análoga, podemos personalizar la **deserialización**. Para esto debemos implementar una función debe ser capaz de manejar un diccionario y retorne un objeto de Python. Esta

función se entregará al parámetros `object_hook` de los métodos `load` y `loads`. En el proceso de deserialización, todo objeto JSON será convertido a un diccionario de Python, y luego pasado a la función `object_hook` para hacer la transformación final.

Por ejemplo, si queremos cargar datos JSON en una lista de tuplas en vez de un diccionario:

```
def funcion_lista_hook(diccionario: dict):
    return [(key, value) for key, value in diccionario.items()]


json_string = '{"nombre": "Luna", "edad": 8, "especie": "Gato",
"ano_nacimiento": 2017}'
datos = json.loads(json_string, object_hook=funcion_lista_hook)

print(datos)
```

```
[('nombre', 'Luna'), ('edad', 8), ('especie', 'Gato'),
('ano_nacimiento', 2017)]
```

O si queremos volver a instanciar las Mascotas de antes:

```
def funcion_mascota_hook(diccionario: dict) -> Mascota:
    mascota = Mascota(diccionario["nombre"], diccionario["edad"],
diccionario["especie"])
    mascota.id_ = diccionario["Mascota_id"]
    return mascota


lista_json_mascotas = [
    '{"Mascota_id": 0, "nombre": "Luna", "edad": 8, "especie": "Gato",
"ano_nacimiento": 2017}',
    '{"Mascota_id": 1, "nombre": "Pepa", "edad": 70, "especie":
"Tortuga", "ano_nacimiento": 1955}',
    '{"Mascota_id": 2, "nombre": "Cachirulo", "edad": 5, "especie":
"Gato", "ano_nacimiento": 2020}',
    '{"Mascota_id": 3, "nombre": "Guillermo", "edad": 0, "especie":
"Gato", "ano_nacimiento": 2025}',
]

lista_objetos_mascotas = []
for json_mascota in lista_json_mascotas:
    objeto_mascota = json.loads(json_mascota,
object_hook=funcion_mascota_hook)
    lista_objetos_mascotas.append(objeto_mascota)

print(lista_objetos_mascotas)
```

```
[<__main__.Mascota object at 0x735294db1550>, <__main__.Mascota object
at 0x735294db16d0>, <__main__.Mascota object at 0x735294db0d10>,
<__main__.Mascota object at 0x735294db1850>]
```

Es importante recordar que **todo objeto de JSON es convertido a un diccionario**, y luego entregado al `object_hook`. Esto tiene implicancias cuando tenemos un JSON con objetos anidados:

```
def funcion_lista_hook(diccionario):
    """Esta función transforma un diccionario en un número"""
    print(f"Me llegó el diccionario: {diccionario}")
    valor = 33 if len(diccionario) > 1 else 42
    print(f"Lo transformaré en {valor}\n")
    return valor


json_string = '{"a": {"b": 1, "c": [2, 3, {}], "d": null, "e": {"f":
true}}}'
datos = json.loads(json_string, object_hook=funcion_lista_hook)

print(datos)
```

```
Me llegó el diccionario: {}
Lo transformaré en 42

Me llegó el diccionario: {'f': True}
Lo transformaré en 42

Me llegó el diccionario: {'b': 1, 'c': [2, 3, 42], 'd': None, 'e': 42}
Lo transformaré en 33

Me llegó el diccionario: {'a': 33}
Lo transformaré en 42

42
```

¿Qué pasó ahí arriba? Lo primero que podemos notar es que el proceso de conversión funciona de adentro hacia afuera, y además que el resultado que entregue `object_hook` se "arrastra" en el proceso.

También se puede ver que las las listas y otros tipos de datos no pasan por el `object_hook`, sino que la función sólo se llama cuando se pasa por un diccionario. Y esta es una de las principales diferencias con cuando se usa `cls`

Errores en serialización

Ocasionalmente puede ocurrir que te aparezcan errores cuando deserializas, esto puede ser porque no se pasaron todos los datos o se corrompió la información.

```
# Falta el ']' del final
json.loads('["Arreglo que no es cerrado"')
```

```
---------------------------------------------------------------------
-----
```

```
JSONDecodeError                           Traceback (most recent call
last)
Cell In[15], line 2
      1 # Falta el ']' del final
----> 2 json.loads('["Arreglo que no es cerrado"')

File /usr/lib/python3.12/json/__init__.py:346, in loads(s, cls,
object_hook, parse_float, parse_int, parse_constant,
object_pairs_hook, **kw)
    341     s = s.decode(detect_encoding(s), 'surrogatepass')
    343 if (cls is None and object_hook is None and
    344         parse_int is None and parse_float is None and
    345         parse_constant is None and object_pairs_hook is None
and not kw):
--> 346     return _default_decoder.decode(s)
    347 if cls is None:
    348     cls = JSONDecoder

File /usr/lib/python3.12/json/decoder.py:337, in
JSONDecoder.decode(self, s, _w)
    332 def decode(self, s, _w=WHITESPACE.match):
    333     """Return the Python representation of ``s`` (a ``str``
instance
    334     containing a JSON document).
    335
    336     """
--> 337     obj, end = self.raw_decode(s, idx=_w(s, 0).end())
    338     end = _w(s, end).end()
    339     if end != len(s):

File /usr/lib/python3.12/json/decoder.py:353, in
JSONDecoder.raw_decode(self, s, idx)
    344 """Decode a JSON document from ``s`` (a ``str`` beginning with
    345 a JSON document) and return a 2-tuple of the Python
    346 representation and the index in ``s`` where the document
ended.
   (...)    350
    351 """
    352 try:
--> 353     obj, end = self.scan_once(s, idx)
    354 except StopIteration as err:
    355     raise JSONDecodeError("Expecting value", s, err.value)
from None

JSONDecodeError: Expecting ',' delimiter: line 1 column 29 (char 28)
```

```
# Falta un ':' entre "mascota_mas_nueva" y "Guillermo"
json.loads('{"mascota_mas_vieja": "Pepa", "mascota_mas_nueva"
"Guillermo"}')
```

```
---------------------------------------------------------------------
-----
JSONDecodeError                           Traceback (most recent call
last)
Cell In[13], line 2
      1 # Falta un ':' entre "mascota_mas_nueva" y "Guillermo"
----> 2 json.loads('{"mascota_mas_vieja": "Pepa", "mascota_mas_nueva"
"Guillermo"}')

File ~/.pyenv/versions/3.12.9/lib/python3.12/json/__init__.py:346, in
loads(s, cls, object_hook, parse_float, parse_int, parse_constant,
object_pairs_hook, **kw)
    341     s = s.decode(detect_encoding(s), 'surrogatepass')
    343 if (cls is None and object_hook is None and
    344         parse_int is None and parse_float is None and
    345         parse_constant is None and object_pairs_hook is None
and not kw):
--> 346     return _default_decoder.decode(s)
    347 if cls is None:
    348     cls = JSONDecoder

File ~/.pyenv/versions/3.12.9/lib/python3.12/json/decoder.py:338, in
JSONDecoder.decode(self, s, _w)
    333 def decode(self, s, _w=WHITESPACE.match):
    334     """Return the Python representation of ``s`` (a ``str``
instance
    335     containing a JSON document).
    336
    337     """
--> 338     obj, end = self.raw_decode(s, idx=_w(s, 0).end())
    339     end = _w(s, end).end()
    340     if end != len(s):

File ~/.pyenv/versions/3.12.9/lib/python3.12/json/decoder.py:354, in
JSONDecoder.raw_decode(self, s, idx)
    345 """Decode a JSON document from ``s`` (a ``str`` beginning with
    346 a JSON document) and return a 2-tuple of the Python
    347 representation and the index in ``s`` where the document
ended.
   (...)    351
    352 """
    353 try:
--> 354     obj, end = self.scan_once(s, idx)
    355 except StopIteration as err:
    356     raise JSONDecodeError("Expecting value", s, err.value)
from None

JSONDecodeError: Expecting ':' delimiter: line 1 column 51 (char 50)
```

```python
# Falta una ',' entre los strings del arreglo
import json
from json import JSONDecodeError

try:
    json.loads('["Luna", "Cachirulo" "Guillermo"]')
except JSONDecodeError:
    print("error intentando deserealizar ...")
```

```
error intentando deserealizar ...
```

Estos son solo ejemplos de leer incorrectamente un archivo o texto, lo importante es recordar el nombre de los errores asociados -en este caso `JSONDecodeError`- para así poder identificar de dónde podría estar proviniendo el error.